SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	V. Yugandhar	Reg. No.:	192472353
Section / Batch:		Date:	17/07/26

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
		GRAND TOTAL:	100 Marks

SECTION A – APPLICATION BASED QUESTIONS

36 A search engine indexing algorithm is defined as: $T(n) = 2T(n/2) + n$. Apply the master Theorem and determine the time complexity. clearly explain the chosen case. Analyze how indexing time grow with the web pages

76 An insurance system uses three nested loops to verify policy details, claim history, and supporting records. Recall how triple nested loops affect complexity, Explain the rate of growth, and determine both the time complexity, explain the rate of growth, the determine both the time complexity and space complexity for this process give in this format c ~~solution code~~ and report for both the quest

Code of Conduct

I V. Yugandhar (192472353) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Yugandhar

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)	3	3	2	3	2	34
Q2 (50 Marks)	3	2	3	3	3	34
					Overall Total (100)	68

Faculty In-charge	Course Coordinator	Head of Department
Signature: [signature]	Signature: ____________	Signature: ____________
Date: 17/7/2026	Date: ____________	Date: ____________

NAME:V.Yugandhar

Reg No:192472353

SUBJECT CODE:CSA0606

SUBJECT NAME: Design and Analysis of Algorithms

---Q3

6. Application: Search Engine Indexing

```
#include <stdio.h>
void indexing(int n) {
    if (n <= 1)
        return;
    indexing(n / 2);
    indexing(n / 2);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            // Indexing operation
        }
    }
}
int main() {
    int n = 8;
    indexing(n);
    printf("Search Engine Indexing Completed\n");
    return 0;
}
```

main.c

```c
#include <stdio.h>

void indexing(int n) {
    if (n <= 1)
        return;

    indexing(n / 2);
    indexing(n / 2);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            // Indexing operation
        }
    }
}

int main() {
    int n = 8;
```

Output

```
Search Engine Indexing Completed


=== Code Execution Successful ===
```

Report

Problem:

Recurrence Relation:

$$T(n) = 2T(n/2) + n^2$$

Master Theorem Analysis:

- $a = 2$
- $b = 2$
- $f(n) = n^2$

Calculate:

$$n^{\log_b a} = n^{\log_2 2} = n$$

Since:

$$f(n) = n^2 = \Omega\left(n^{1+\epsilon}\right) \ (\epsilon = 1)$$

and the regularity condition holds,

Master Theorem Case 3 applies.

Time Complexity:

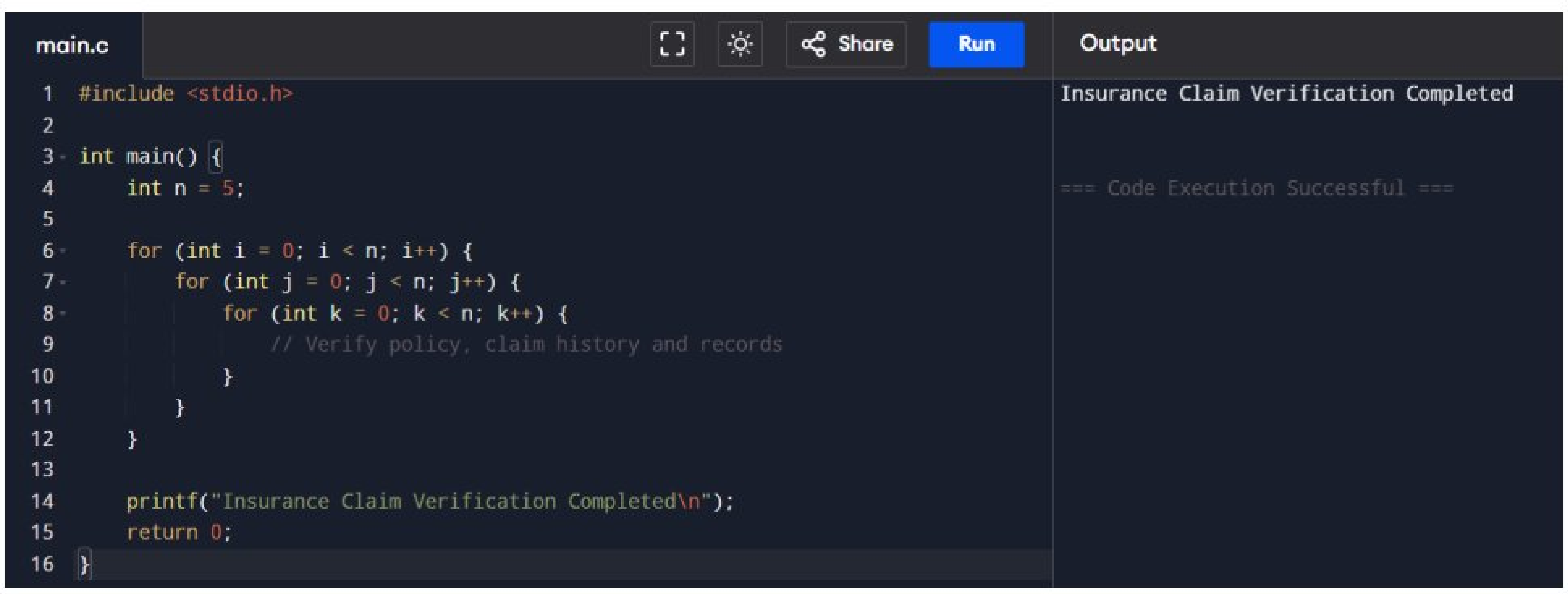
$$O\left(n^2\right)$$

Explanation:

- Recursive calls divide the data into two halves.
- The indexing work at each level is n^2, which dominates the recursion.
- Therefore, indexing time grows **quadratically** as the number of web pages increases.

Q76. Application: Insurance Claim Verification System

```
#include <stdio.h>

int main() {

    int n = 5;

    for (int i = 0; i < n; i++) {

        for (int j = 0; j < n; j++) {

            for (int k = 0; k < n; k++) {

            }

        }

    }

    printf("Insurance Claim Verification Completed\n");

return 0;

}
```



Report

Problem:
The system verifies:

- Policy details
- Claim history
- Supporting records

using **three nested loops**.

Complexity Analysis:

Each loop executes **n** times.

Total operations:

$$n \times n \times n = n^3$$

Time Complexity:

$$O\left(n^3\right)$$

Space Complexity:

Only loop variables are used.

$$O(1)$$

Rate of Growth:

- Triple nested loops increase execution time **cubically**.
- If the number of records doubles, the execution time increases by approximately **8 times** ($2^3 = 8$).
- Hence, the verification process becomes significantly slower for very large datasets.